This article was downloaded by: [LIU Libraries]
On: 02 May 2014, At: 08:10
Publisher: Taylor & Francis
Informa Ltd Registered in England and Wales Registered Number: 1072954 Registered office: Mortimer House, 37-41 Mortimer Street, London W1T 3JH, UK

Optimization Methods and Software
Publication details, including instructions for authors and subscription information:
http://www.tandfonline.com/loi/goms20

Solving mathematical programs with complementarity constraints as nonlinear programs

Roger Fletcher [a] & Sven Leyffer, [b]

[a] Department of Mathematics , University of Dundee , 21 Perth Road, Dundee, DD1 4HN, UK

[b] Mathematics and Computer Science Division , Argonne National Laboratory , 9700 South Cass Avenue, Argonne, Illinois, 60439-4844, USA

Published online: 13 May 2010.

To cite this article: Roger Fletcher & Sven Leyffer, (2004) Solving mathematical programs with complementarity constraints as nonlinear programs , Optimization Methods and Software, 19:1, 15-40, DOI: 10.1080/10556780410001654241

To link to this article: http://dx.doi.org/10.1080/10556780410001654241

PLEASE SCROLL DOWN FOR ARTICLE

Taylor & Francis makes every effort to ensure the accuracy of all the information (the "Content") contained in the publications on our platform. However, Taylor & Francis, our agents, and our licensors make no representations or warranties whatsoever as to the accuracy, completeness, or suitability for any purpose of the Content. Any opinions and views expressed in this publication are the opinions and views of the authors, and are not the views of or endorsed by Taylor & Francis. The accuracy of the Content should not be relied upon and should be independently verified with primary sources of information. Taylor and Francis shall not be liable for any losses, actions, claims, proceedings, demands, costs, expenses, damages, and other liabilities whatsoever or howsoever caused arising directly or indirectly in connection with, in relation to or arising out of the use of the Content.

This article may be used for research, teaching, and private study purposes. Any substantial or systematic reproduction, redistribution, reselling, loan, sub-licensing, systematic supply, or distribution in any form to anyone is expressly forbidden. Terms &

Conditions of access and use can be found at http://www.tandfonline.com/page/terms-and-conditions

Downloaded by [LIU Libraries] at 08:10 02 May 2014

Optimization Methods and Software
Vol. 19, No. 1, February 2004, pp. 15–40

Taylor & Francis
Taylor & Francis Group

SOLVING MATHEMATICAL PROGRAMS WITH COMPLEMENTARITY CONSTRAINTS AS NONLINEAR PROGRAMS

ROGER FLETCHER[a,*] and SVEN LEYFFER[b,†,‡]

[a]*Department of Mathematics, University of Dundee, 21 Perth Road, Dundee, DD1 4HN, UK;*
[b]*Mathematics and Computer Science Division, Argonne National Laboratory, 9700 South Cass Avenue, Argonne, Illinois 60439-4844, USA*

(Received 20 August 2002; Revised 22 September 2003; In final form 19 November 2003)

Dedicated to Professor Jochem Zowe

We consider solving mathematical programs with complementarity constraints (MPCCs) as nonlinear programs (NLPs) using standard NLP solvers. This approach is appealing because it allows existing off-the-shelf NLP solvers to tackle large instances of MPCCs. Numerical experience on MacMPEC, a large collection of MPCC test problems is presented. Our experience indicates that sequential quadratic programming (SQP) methods are very well suited for solving MPCCs and at present outperform interior-point solvers both in terms of speed and reliability. All NLP solvers also compare very favorably to special MPCC solvers on tests published in the literature.

Keywords: MPCC; Complementarity constraints; Nonlinear programming; Sequential quadratic programming; Interior-point methods

AMS-MSC2000: 90C30; 90C33; 90C55; 49M37; 65K10

Downloaded by [LIU Libraries] at 08:10 02 May 2014

1 INTRODUCTION

We consider mathematical programs with complementarity constraints (MPCCs) of the form

$$\begin{aligned} \text{minimize} \quad & f(z) \\ \text{subject to} \quad & c_{\mathcal{E}}(z) = 0 \\ & c_{\mathcal{I}}(z) \geq 0 \\ & 0 \leq z_1 \perp z_2 \geq 0, \end{aligned} \tag{1}$$

where $z = (z_0, z_1, z_2)$, $z_0 \in \mathbb{R}^n$ are the control and $(z_1, z_2) \in \mathbb{R}^{2p}$are the state variables. The equality constraints $c_i(z) = 0$, $i \in \mathcal{E}$ are abbreviated as $c_{\mathcal{E}}(z) = 0$ and similarly, $c_{\mathcal{I}}(z) \geq 0$ are

* fletcher@maths.dundee.ac.uk
† Corresponding author. Tel.: (+1) (630) 252-4698; E-mail: leyffer@mcs.anl.gov
‡ This work was carried out while the second author was at the University of Dundee.

ISSN 1055-6788 print; ISSN 1029-4937 online © 2004 Taylor & Francis Ltd
DOI:10.1080/10556780410001654241

the inequality constraints. Problems of this type arise frequently in applications, see Ref. [1–3]. Problem (1) is a subclass of more general mathematical programs with equilibrium constraints.

Clearly, any MPCC with a more general complementarity condition such as for instance $0 \leq y \perp F(x, y) \geq 0$, can be written in the form (1) by introducing slack variables. It is easy to show that the reformulated MPCC has the same properties (such as constraint qualifications or second order conditions) as the original MPCC. In this sense, nothing is lost by introducing slacks.

One interesting way of solving Eq. (1) is to consider its equivalent nonlinear programming NLP formulation,

$$\begin{aligned} \text{minimize} \quad & f(z) \\ \text{subject to} \quad & c_{\mathcal{E}}(z) = 0 \\ & c_{\mathcal{I}}(z) \geq 0 \\ & z_1 \geq 0 \\ & z_2 \geq 0 \\ & z_1^{\mathrm{T}} z_2 \leq 0, \end{aligned} \tag{2}$$

and solve Eq. (2) with existing nonlinear program (NLP) solvers. This article introduces a new collection of MPCC test problems, MacMPEC [4], and demonstrates that a large class of MPCCs can in fact be solved successfully as NLPs.

The article is organized as follows. Section 2 reviews recent theoretical advances which are relevant to the NLP approach as well as previous numerical experience. Section 3 describes a new AMPL interface to the NLP solver `filterSQP` that enables the sequential quadratic programming (SQP) method to solve MPCCs exploiting their special structure. Section 4 introduces a new test problem library with over 100 MPCCs and describes some of their characteristics. Section 5 presents our numerical experience with some large-scale NLP solvers.

Downloaded by [LIU Libraries] at 08:10 02 May 2014

2 MPCC BACKGROUND

This section summarizes some background material that is relevant to the NLP approach for solving MPCCs as Eq. (2). We also comment briefly on previous numerical experience.

2.1 Theoretical Background to MPCCs

It is easy to see that MPCCs violate the Mangasarian Fromovitz constraint qualification (MFCQ), see Refs. [5,6]. This failure of MFCQ implies that the multiplier set is unbounded and this has been taken as an argument against the use of NLP techniques for solving MPCCs.

On the other hand, it is shown in Ref. [7], that strong stationarity [6] of Eq. (1) is equivalent to the existence of Karush–Kuhn–Tucker (KKT) conditions multipliers for the equivalent NLP (2), i.e. the existence of multipliers $\mu := (\lambda, \nu_1, \nu_2, \xi)$ such that $\lambda_{\mathcal{I}} \geq 0$, $\nu_1, \nu_2 \geq 0$, $\xi \geq 0$ and

$$\nabla f(z) - \nabla c(z)^{\mathrm{T}} \lambda - \begin{pmatrix} 0 \\ \nu_1 \\ 0 \end{pmatrix} - \begin{pmatrix} 0 \\ 0 \\ \nu_2 \end{pmatrix} + \xi \begin{pmatrix} 0 \\ z_2 \\ z_1 \end{pmatrix} = 0$$

$$c_{\mathcal{E}}(z) \geq 0 \quad \text{and} \quad c_{\mathcal{I}}(z) \geq 0$$

$$z_1 \geq 0 \quad \text{and} \quad z_2 \geq 0$$

$$
\begin{gathered}
z_1^{\mathrm{T}} z_2 \leq 0 \\
c(z)^{\mathrm{T}}\lambda = 0, \quad z_1^{\mathrm{T}}\nu_1 = 0, \quad z_2^{\mathrm{T}}\nu_2 = 0, \quad \xi z_1^{\mathrm{T}} z_2 = 0
\end{gathered}
\tag{3}
$$

Note that the complementarity condition $\xi z_1^{\mathrm{T}} z_2 = 0$ is implied by feasibility of z_1, z_2.

These KKT conditions also indicate how the unboundedness of the multiplier set arises. For a given set of multipliers, define $\pi_1 = \nu_1 - \xi z_2$ and $\pi_2 = \nu_2 - \xi z_1$, then any other $\hat{\nu}_1, \hat{\nu}_2, \hat{\xi} \geq 0$ that keeps (π_1, π_2) constant also satisfies these conditions. The multipliers π_1, π_2 are in fact the multipliers of the relaxed NLP, see Ref. [7].

It is shown in Ref. [7] that there exist a certain minimal multiplier vector, defined by

$$
\xi = \max\left\{0, \max_{i \in \mathcal{Z}_2^{\perp}} \frac{-\hat{\nu}_{1i}}{z_{2i}^*}, \max_{i \in \mathcal{Z}_1^{\perp}} \frac{-\hat{\nu}_{2i}}{z_{1i}^*}\right\}. \tag{4}
$$

The nonzero components of this multiplier correspond to a linearly independent set of constraint normals (provided an MPCC Linear Independence Constraint Qualification (MPCC-LICQ) holds). Thus, the multiplier set is a ray whose base is the minimal or basic multiplier vector. It can be shown that SQP methods converge quadratically to this basic multiplier, provided all QPs remain consistent. The assumption that all QPs remain consistent is shown to be satisfied for a certain class of MPCCs with vertical complementarity constraints, such as

$$
\begin{aligned}
&\text{minimize} && f(z) \\
&\text{subject to} && 0 \leq G(z) \perp H(z) \geq 0.
\end{aligned}
$$

The most important consequence of the failure of MFCQ is the fact that a QP approximation to the equivalent NLP (2) can become inconsistent arbitrarily close to a strongly stationary point. This has important implications for the design of SQP methods for MPCCs. Anitescu [8] shows that an SQP method with elastic mode (such as `snopt` [9]) converges globally for MPCCs. An alternative is to relax the linearization of the complementarity constraint and this idea is adopted here together with a restoration phase.

An important ingredient in the success of NLP solvers has been the replacement of the usual complementarity condition $z_1^{\mathrm{T}} z_2 = 0$ by the equivalent condition $z_1^{\mathrm{T}} z_2 \leq 0$. Without this relaxation, SQP methods in general, cannot, converge quadratically near a strongly stationary point. Consider for example

$$
\begin{aligned}
&\text{minimize} && z_1 + z_2 \\
&\text{subject to} && 0 \leq z_1 \perp z_2 \geq 0,
\end{aligned}
$$

with solution at $z^* = (0, 0)$. Writing the complementarity condition as $z_1 z_2 = 0$ and starting at any point with $z_1 = z_2$, e.g. $z^{(0)} = (1, 1)$ results in SQP converging to z^* at linear rate (the iterates are $z^{(k)} = (1/2^k) z^{(0)}$). The reason for this slow convergence is that by forcing the iterates to lie on the linearization of $z_1 z_2 = 0$, complementarity is only reduced linearly at each iteration. The multiplier ξ diverges to infinity in this case. In contrast, using $z_1 z_2 \leq 0$, gives convergence in 1 iteration, allowing SQP to identify the correct active set rapidly.

2.2 Previous Numerical Experience with MPCCs

Numerical experience with Eq. (2) in the past has been rather disappointing. Bard [10] reports failure on 50–70% of some bilevel problems for a gradient projection method. Conn *et al.* [11]

Downloaded by [LIU Libraries] at 08:10 02 May 2014

and Ferris and Pang [1] attribute certain failures of `lancelot` for `LUBRIF` to the fact that the problem contains a complementarity constraint. However, it turns out that this problem minimizes the complementarity error. Consequently, complementarity cannot be the cause for this failure.

The most obvious approach to smoothing is to replace $z_1^{\mathrm{T}} z_2 \leq 0$ by

$$z_1^{\mathrm{T}} z_2 \leq \tau_k,$$

and solve a sequence of NLPs, driving τ_k to zero, see Ferris and Kanzow [12]. This approach is also analyzed by Scheel and Scholtes [6]. An alternative is to penalize the complementarity constraint, solving a sequence of NLPs where the objective is modified as

$$\text{minimize} \quad f(z) + \nu_k z_1^{\mathrm{T}} z_2,$$

for a sequence of increasing penalty parameters $\nu_k > 0$. This approach is also analyzed in Anitescu [8], who shows that under reasonable assumptions, a finite $\nu_k > 0$ is sufficient to give an exact penalization. Ferris and Tin Loi [13] conduct numerical experiments comparing both approaches.

Another interesting idea of Facchinei *et al.* [15] is to replace the complementarity constraint by the smoothed min-function,

$$\psi_\mu(z_{1i}, z_{2i}) = \sqrt{(z_{1i} - z_{2i})^2 + 4\mu} - z_{1i} - z_{2i} = 0,$$

where $\mu > 0$ is a parameter. For $\mu = 0$, it can be shown that $\psi_0(a, b) = -2\min(a, b)$. The resulting NLP is differentiable with bounded gradients, as long as $(z_{1i}, z_{2i}, \mu_k) \neq 0$. In Ref. [14], various algorithms based on solving a sequence of NLPs for decreasing $\mu \to 0$ (inexactly) are suggested and their convergence properties are investigated.

Jiang and Ralph [15] propose two smooth SQP methods for solving MPCCs. They replace the complementarity constraint by the smoothed Fischer–Burmeister functional

$$\psi_\mu(z_{1i}, z_{2i}) = \sqrt{z_{1i}^2 + z_{2i}^2 + \mu} - z_{1i} - z_{2i} = 0,$$

and propose two methods for solving the resulting NLP. The first method applies SQP to the problem where μ is reduced at every iteration, while the second method adds the equation $\exp(\mu) - 1 = 0$ to the NLP and then applies SQP. This equation ensures that in the limit, $\mu = 0$. In order to overcome the problems caused by infeasible QP subproblems, they apply a special ℓ_1 exact penalty function approach. Fukushima *et al.* [16] propose a similar algorithm based on parametrically reducing μ at every iteration of SQP and report a numerical comparison to the penalty interior-point algorithm (PIPA) [2].

The PIPA aims to maintain $z_1 > 0$ and $z_2 > 0$ while using an SQP approach for solving NLP (2). Unfortunately, PIPA can converge to nonstationary points under certain circumstances [19].

The implicit programming approach of Outrata *et al.* [3] (see also Dirkse and Ferris [18]) solves the lower level complementarity problem for fixed controls and computes sensitivities of the controls. At the upper level, a nonsmooth optimization problem is then solved using bundle trust-region techniques.

Recently, Dirkse *et al.* [19] have investigated automatic reformulation techniques for MPCCs formulated in GAMS. They considered 23 smooth reformulations and report numerical experience on a range of large MPCCs.

Downloaded by [LIU Libraries] at 08:10 02 May 2014

3 `filtermpec`: AN AMPL SOLVER INTERFACE

This section describes a SQP algorithm which has been adapted to solve MPCCs. The SQP method used here is a trust-region SQP algorithm, `filterSQP` [20]. It uses a 'filter' to promote global convergence. A filter accepts a trial point whenever the objective or the constraint violation is improved compared to all previous iterates. Much of the comments in this section would also apply to other generic SQP methods.

We have written an AMPL [21] interface for our NLP approach to MPCCs. This interface takes the MPCC information generated by AMPL and passes an equivalent NLP in form (2) to the NLP solver.

In many applications, the complementarity constraint has the more general from

$$l_G \leq G(z) \leq u_G \perp l_F \leq F(z) \leq u_F, \tag{5}$$

where exactly two bounds are finite for each component of G and F. Clearly, if $l_{G_i} = u_{G_i}$ then the complementarity simplifies to a standard equation and a free constraint on the corresponding component of F. This usually poses no difficulties for NLP solvers. In our experience, SQP and interior-point methods benefit from reformulating general complementarity constraints by introducing slacks, so that complementarity occurs only between variables. There is also some theoretical evidence which supports this idea [7]. This is the main motivation behind introducing a special AMPL interface.

The complementarity constraint, which AMPL presents to the solver is of the form

$$l_{y_i} \leq y_i \leq u_{y_i} \perp l_{F_i} \leq F_i(x, y) \leq u_{F_i}, \tag{6}$$

where exactly two bounds are finite. Note that AMPL re-arranges more complex complementarity constraints in such a way that one side always involves only a simple variable bound.

If $l_{y_i} = u_{y_i}$ or $l_{F_i} = u_{F_i}$, then the complementarity condition (6) is replaced by an equality constraint and a free variable (or equation). Otherwise, slacks are introduced, so that Eq. (6) becomes

$$l_{y_i} \leq y_i \leq u_{y_i} \perp l_{F_i} \leq s_i \leq u_{F_i} \quad \text{and} \quad s_i = F_i(x, y). \tag{7}$$

If $F_i(x, y)$ is also a simple bound constraint, then no slacks are needed and the interface does not introduce them in this situation.

If $l_{y_i} > -\infty$ and $l_{F_i} > -\infty$, then the complementarity can be expressed as

$$(y_i - l_{y_i})(s_i - l_{F_i}) \leq 0. \tag{8}$$

Similar expressions are readily derived for all the other cases. The interface adds a single nonlinear equation, summing all complementarity conditions to the NLP. The modifications to the constraint, gradient and Hessian evaluation are relatively minor and the additional overhead is negligible, if a sparse matrix storage scheme is used (as in our case). The interface also contains some pre- and post-processing routines for setting up data structures and evaluating/printing the results. The whole package is available from the authors by request.

The interface currently does not handle complementarity constraints of the form

$$y_i \perp l_{F_i} \leq F_i(x, y) \leq u_{F_i},$$

Downloaded by [LIU Libraries] at 08:10 02 May 2014

unless $l_{F_i} = u_{F_i}$ in which case an equation is detected. If $l_{F_i} < u_{F_i}$ then it is not clear, how best to model this type of complementarity condition within an NLP, using e.g. (8). Note that this situation can always be modeled with two complementarity conditions.

Of course, it would have been possible to solve MPCCs formulated as nonlinear AMPL models without the `complements` command. However, the MPCC interface has several advantages. The use of the `complements` structure makes the models more transparent. Details such as the addition of slacks are hidden from the user. In addition, it becomes possible to generate various NLP representations from the same MPCC model, see Ref. [22]. Finally, AMPL provides some pre-solve facilities for MPCCs [23]. By using the `complements` command this can be exploited, whereas the same pre-solve is not available for MPCCs written as NLPs such as Eq. (2).

It has been shown in Ref. [7], that linearizations of the complementarity constraint can cause the QP approximations to Eq. (2) to become inconsistent arbitrarily close to a strongly stationary point. To avoid the difficulties associated with this, a simple heuristic has been proposed. Let $d = (d_0, d_1, d_2)$ be the SQP step, then we perturb the linearization of $z_1^{\mathrm{T}} z_2 \leq 0$ to

$$z_1^{(k)^{\mathrm{T}}} z_2^{(k)} + z_2^{(k)^{\mathrm{T}}} d_1 + z_1^{(k)^{\mathrm{T}}} d_2 \leq \delta \left(z_1^{(k)^{\mathrm{T}}} z_2^{(k)} \right)^{1+\kappa}, \tag{9}$$

where $0 < \delta, \kappa < 1$ are constants that define the perturbation of the right hand side. Note that this perturbation only gives linear convergence for $\kappa = 0$.

Additional solver options have been added to `filtermpec` to handle MPCCs. These are

- `initial_slack`
  If set to 0 (default), then the slack variables in Eq. (7) are initialized to 0. If set to 1, then the slacks are initialized as

  $$s_i = \max \left\{ 0, F_i(x^{(0)}, y^{(0)}) \right\},$$

  where $(x^{(0)}, y^{(0)})$ is the initial point. Preliminary tests indicate no significant advantages of one option over the other, though initializing slacks in this way can be beneficial in some cases and is important to hot-start MPCCs.
- `compl_tol`
  This is the value, μ, of the right hand side of $z_1^{\mathrm{T}} z_2 \leq \mu$. This allows the solver to be used in smoothed mode for a decreasing sequence of μ. Its default value is `0.0E0`. This option is useful in finding initial points which satisfy the constraints apart from complementarity in case the NLP reformulation is inconsistent.
- `delta, kappa`
  These are the values of δ and κ in Eq. (9). The default values which appear to work well are `delta = 1.0E-2` and `kappa = 1.0`.

4 MacMPEC: AN MPCC TEST PROBLEM LIBRARY

This section describes an MPCC problem library for benchmarking MPCC and NLP solvers, called MacMPEC. All models are written in AMPL and are available at www.mcs.anl.gov/∼leyffer/MacMPEC/. The library also contains extensive comments and references for all models as well as plots of some of the solutions. Some of the models are novel and are described in more detail toward the end of the section.

The motivation for collecting this test set is two-fold. Firstly, we are interested in testing our own ideas on a large collection of test problems. Secondly, we hope that the ready availability

Downloaded by [LIU Libraries] at 08:10 02 May 2014

of this library of problems motivates research into numerical aspects of MPCCs and leads to the development of algorithm with both good theoretical properties and practical performance.

4.1 Classification Scheme for MPCCs

To maintain a large collection of test problems and draw meaningful conclusions from numerical tests, it is important to suitably classify the test library. A standard NLP classification scheme has been extended to cover MPCC test problems. Each problem is identified by a string generalized form [4].

$$\texttt{XXX} - \texttt{XX} - \texttt{CCC} - \texttt{n} - \texttt{m} - \texttt{p}$$

consisting of three groups of characters and three integers. The first two groups of characters describe the general aspect of the MPCC, the third group describes the aspect of the complementarity constraint and the integers represent the problem dimensions. Each group of characters and integers is now described in turn.

The first group of characters is identical to the scheme used in Ref. [24] and describes the type and regularity of the objective and constraints. The second group of characters describes the origin of the model (A = academic, M = modeling exercise and R = real application) and whether or not the lower level variables appear in the general constraint (Y = yes, N = no). The third group of characters describes the type of the complementarity constraint and follows Ref. [25]. It is MCP for (general) mixed complementarity problems, LMCP for linear mixed complementarity problems, NCP for nonlinear complementarity problems, LCP for linear complementarity problem and NLP if the lower level problem stems from optimality conditions of a nonlinear program. Finally, the integers `n-m-p` represent the number of variables, constraints (excluding complementarity constraints) and complementarity conditions.

Currently, the library contains 137 MPCCs. The lower level complementarity constraint arise in various forms: 83 are LCPs, 15 are NCPs and 39 are KKT conditions of some lower level optimization problem. The problem sizes are summarized in Table I which shows the quantile distribution of the number of variables, constraints and complementarity constraints. Appendix A gives detailed characteristics of the MPCC test problems.

There is a number of novel models in MacMPEC and these are described below.

4.2 Description of Selected Models

Problems `hs044-i`, `liswet1-*` and `portfl*` are inverse QP problems which give rise to QPECs, following the idea by Stefan Scholtes. These problems are obtained by taking the solution to the original QP and then perturbing some of the QP data, introducing controls. The inverse problem then arises by seeking the controls for which the solution of the perturbed QP is closest to the original solution. The complementarity constraints are the complementary slackness conditions of the perturbed QP.

TABLE I Summary of MPCC test problem characteristics

Quantile	*No. of variables*	*No. of constraints*	*No. of complementarities*
1–9	34	58	66
10–99	54	34	29
100–999	38	41	35
1000–9999	11	4	7

Downloaded by [LIU Libraries] at 08:10 02 May 2014

The models `water-*` are two small water distribution systems. The smaller model is derived from a mixed integer nonlinear programming GAMS model, which includes binary variables to model the pressure loss equation along each arc,

$$h_i - h_j = \frac{c_{ij}}{d_{ij}^p} q_{ij}^2 \operatorname{sign}(q_{ij}),$$

where the variables are h_i, the pressure at node i, q_{ij}, the flow along arc ij, d_{ij}, the diameter of the arc. The parameters are c_{ij} and p. This nonsmooth equation is usually modeled by introducing binary variables $z_{ij} \in \{0, 1\}$ and splitting q_{ij} into its positive, $q_{ij}^+ \geq 0$ and negative, $q_{ij}^- \geq 0$ part, giving rise to the system

$$\begin{aligned} & z_{ij} \in \{0, 1\} \\ & q_{ij}^+ \leq M z_{ij} \quad \text{and} \quad h_i - h_j = \frac{c_{ij}}{d_{ij}^p}((q_{ij}^+)^2 + (q_{ij}^-)^2), \\ & q_{ij}^- \leq M(1 - z_{ij}) \end{aligned}$$

where $M > 0$ is some large constant, see www.gams.com/modlib/libhtml/waterx.htm

In MacMPEC the binary variables are replaced by a disjunction,

$$0 \leq q_{ij}^+ \perp q_{ij}^- \geq 0 \quad \text{and} \quad h_i - h_j = \frac{c_{ij}}{d_{ij}^p}((q_{ij}^+)^2 + (q_{ij}^-)^2).$$

The use of the disjunction removes the need for binary variables. The new model `water-FL` is a larger network than the original model and represents the water distribution network of `Flatland`.

5 NUMERICAL RESULTS

This section presents the results of a numerical experiment in which a range of NLP solvers has been tested on a collection of MPCCs formulated as NLPs (2). Details of the test problems can be found in Appendix A. We also compare the performance of the NLP solvers to some MPCC solvers reported in the literature. In this section, we only summarize the results; detailed results for all solvers and runs can be found in Appendix B. The following large-scale NLP solvers on NEOS [26] are included in the test.

Solver	*Description and references*
`filtermpec`	SQP trust-region method; global convergence through use of a filter; MPCC AMPL interface; exact Hessian; feasibility restoration [20,27].
`knitro`	Trust-region interior-point algorithm; step decomposed into normal and tangential step; special rules for trust-region adjustment [28].
`loqo`	Primal–dual interior-point method; line-search to induce global convergence; exact Hessian [29].
`snopt`	SQP line-search method; augmented Lagrangian merit function; quasi-Newton Hessian, inertia control; elastic mode for inconsistent QP subproblems.

Downloaded by [LIU Libraries] at 08:10 02 May 2014

`filtermpec` is the only solver that uses a special MPCC interface to AMPL. All other codes use an equivalent NLP formulation, and we have made sure that the problems are identical in both cases (have the same number of variables and constraints).

5.1 Failures of NLP Solvers

All NLP solvers assume some kind of constraint qualification in order to guarantee convergence. Therefore one cannot expect NLP solvers to converge in the absence of an MFCQ. This section examines the type and nature of failures of the four solvers.

We consider the following two classes of failures of NLP solvers.

1. *Catastrophic failures* are runs, where a solver failed to converge within major 1000 iterations or the trust-region or step became too small at a nonstationary point. These are failures of the algorithm which are likely to be due to the absence of MFCQ.
2. *Soft failures* refer to instances, where a solver terminated with a message that the problem is locally inconsistent. Since all solvers are local methods, they cannot be expected to find a feasible point. This type of failure has (most likely) nothing to do with the absence of MFCQ.

Table II shows the number of catastrophic failures, locally inconsistent problems and local optimal solutions for each solver. In the context of evaluating the suitability of NLP solvers for solving MPCCs, we view only the catastrophic failures as problematic.

These experiments represent a snap-shot of the performance of NLP solvers. Currently, there is much activity devoted at improving the performance of interior-point methods for MPECs, see Refs. [30,31], both report significant performance improvements. There are surprisingly few catastrophic failures for the SQP methods, making these solvers very robust choices for solving MPCCs. Even the number of termination at infeasible points is small. Note, in addition, that `filtermpec` can be shown to have found local infeasibilities where the complementarity error $z_1^{\mathrm{T}} z_2$ is relatively small (this can be seen in Appendix A).

On the other hand, interior-point methods are not as robust as SQP methods for solving MPCCs. Nevertheless, they solve over 85% of the problems, which is still significantly more than that was observed in early tests [10] involving a projected gradient solver.

It is of interest, to investigate whether the failures of `loqo` and `knitro` are solely due to the fact that the problems violate MFCQ. In Tables III and IV, we give the reason for failure for each failed MPCC and also show how the solver performed when the complementarity constraint is replaced by

$$z_1^{\mathrm{T}} z_2 \leq 10^{-2}. \tag{10}$$

TABLE II Number and type of failures for NLP solvers (out of 137 problems)

	Number of failures	
Solver	*Catastrophic*	*Infeasible*
`filtermpec`	0	7
`knitro`	19	6
`loqo`	20	0
`snopt`	1	4

Downloaded by [LIU Libraries] at 08:10 02 May 2014

TABLE III Details of loqo failures after replacing complementarity by $z_1^{\mathrm{T}} z_2 \leq 10^{-2}$

Problem	ξ	*# iter*	*Outcome*
design-cent-4	2.383E−05	1000	Iteration limit
incid-set1-32	−1.563E−13	121	Optimal solution
incid-set1c-32	−8.783E+08	1000	Iteration limit
incid-set2-16	−2.833E−09	81	Optimal solution
incid-set2-32	−2.739E−11	151	Optimal solution
incid-set2c-32	−4.121E+08	1000	Iteration limit
pack-rig2-16	−1.070E−02	1000	Iteration limit
pack-rig2-32	−1.152E−01	1000	Iteration limit
pack-rig2c-16	−2.583E−02	1000	Iteration limit
pack-rig2c-32	−8.469E−05	1000	Iteration limit
pack-rig2p-32	−5.011E−278	435	Dual infeasible
portfl1	−8.119E−09	45	Optimal solution
portfl3	−5.628E−07	52	Optimal solution
portfl4	−4.530E−08	53	Optimal solution
ralphmod	−6.078E+10	1000	Iteration limit
scholtes4	−1.000E+01	13	Optimal solution
tollmpec	1.258E+07	1000	Iteration limit
tollmpec1	−1.284E+07	1000	Iteration limit
water-FL	−7.510E+10	1000	Iteration limit
water-net	−4.079E+10	584	Primal/dual infeasible

Downloaded by [LIU Libraries] at 08:10 02 May 2014

This experiment will give an indication as to whether the failures are due to the absence of MFCQ. The only failure of snopt (incid-set1-16) is solved successfully in 10 iterations using the relaxed formulation, and gives zero complementarity at the end.

All loqo failures are due to reaching the iteration limit (1000) on the original NLP formulation of the MPCC. After replacing the complementarity constraint by Eq. (10), loqo still fails on 11 out of 20 NLP problems. Table III shows that loqo's failures are not solely due to the fact that the MPCCs violate MFCQ. Instead, other factors clearly play a role. One such factor might be the difficulty in finding feasible points or terminating satisfactorily, if no such point exists.

knitro's failures on the MPCC set are also mostly due to reaching the iteration limit (1000). The two exceptions are gnash14, which terminates after trying to evaluate the objective outside the admissible range and water-FL which terminates with the trust-region radius becoming smaller than 10 times the machine precision. We observe, that knitro manages to solve most of these NLPs (13 out of 19) with the relaxed formulation (10). Six problems still fail and we suspect this might be because the problems are locally inconsistent.

One exception in knitro's failures is dempe. The problem is given by

$$\begin{aligned} \underset{x,\,w,\,z}{\text{minimize}} \quad & (x-3.5)^2 + (z+4)^2 \\ \text{subject to} \quad & z - 3 + 2zw = 0 \\ & 0 \leq x - z^2 \perp w \geq 0, \end{aligned}$$

and the solution to the lower level problem is readily computed as

$$z(x) = \begin{cases} 3 & \text{for } x \geq 9 \\ \sqrt{x} & \text{for } 0 \leq x \leq 9 \\ \text{infeasible} & \text{for } x < 0. \end{cases}$$

TABLE IV Details of `knitro` failures after replacing complementarity by $z_1^T z_2 \leq 10^{-2}$

Problem	ξ	*#iter*	*Outcome*
`dempe`	3.325E−03	143	Optimal solution
`gnash14`	−1.705E−08	52	Optimal solution
`incid-set1-16`	1.843E−07	33	Optimal solution
`incid-set1-32`	1.635E−07	56	Optimal solution
`incid-set1c-16`	7.495E−08	32	Optimal solution
`incid-set1c-32`	1.339E−07	20	Optimal solution
`incid-set2-16`	−1.164E−07	81	Optimal solution
`incid-set2-32`	−5.450E−07	124	Optimal solution
`incid-set2c-16`	−2.692E−06	22	Optimal solution
`incid-set2c-32`	−1.351E−05	28	Optimal solution
`pack-comp1-32`	1.826E−05	23	Optimal solution
`pack-rig1-32`	1.396E−04	34	Optimal solution
`pack-rig1p-32`	1.390E−04	33	Optimal solution
`pack-rig2-16`	1.764E−03	1000	Iteration limit
`pack-rig2-32`	3.748E−04	1000	Iteration limit
`pack-rig2c-16`	3.392E−04	1000	Iteration limit
`pack-rig2c-32`	1.823E−03	1000	Iteration limit
`water-FL`	−53.6972	37	Trust-region got too small
`water-net`	−2.08034	1000	Iteration limit

Substituting $z(x)$ into the objective function, it can be shown that there exist two local minima at $x = 1$ and $x = 0$. However, for $x = 0$, the lower level problem contains the constraint $z^2 \leq 0$ which violates MFCQ and there are no multipliers $w \geq 0$ such that $z - 3 + 2zw = 0$ is satisfied. Thus, this limit is not even a B-stationary point. In this sense, `dempe` is a pathological example. For this problem, `knitro` is the only solver that converges to the global minimum at $x = 0$. All other solvers converge to the local minimum at $x = 1$. Thus the failure of `knitro` should be regarded as a failure of the problem formulation, rather than the method.

This section clearly shows that interior-point methods can experience difficulties in solving NLPs which are not related to the complementarity constraint. The challenge for developers of interior-point methods is to match the performance obtained without the complementarity constraint in the NLP formulation of the MPCCs.

5.2 Comparison of NLP Solvers

This section summarizes the numerical comparison of different NLP solvers for solving MPCCs. Detailed results can be found in Appendix B. The solvers are compared using performance plots of Dolan and Moré [32]. For a solver s, we plot

$$\log_2\left(\frac{\#\text{iter}(s, p)}{\text{best_iter}(p)}\right), \quad \forall p \in \text{ problem},$$

where $\#\,\text{iter}(s, p)$ is the number of iterations solver s took on problem p and $\text{best_iter}(p)$ is the smallest number of iteration any solver took. This can be interpreted as a probability distribution that a given solver is at worse x times slower than the best. We also use the CPU time as a performance measure.

We use both number of iterations and CPU time as performance measures, since the former allows comparison between similar solvers excluding the effect of fast linear algebra. The CPU time comparison on the other hand gives an indication how different solver classes compare.

Downloaded by [LIU Libraries] at 08:10 02 May 2014

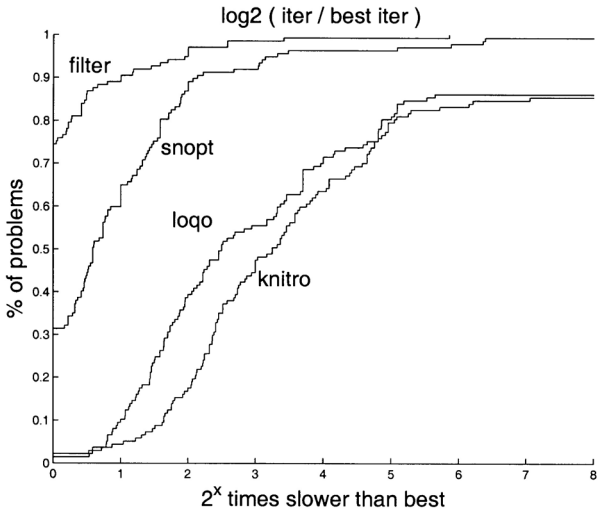

FIGURE 1 Performance plot (iteration).

Figures 1 and 2 show the performance plots for the iteration count and CPU time, respectively. For a catastrophic failure, we set # iter$(s, p) =$ 1E6 and CPU$(s, p) =$ 1E8. This will ensure that the percentage of failures appears as a horizontal asymptotic at the right of the performance plots. Infeasibility failures are not penalized in this way, as they constitute ordinary exits from the NLP solver.

Both in terms of iteration count and CPU time, the SQP methods are clear winners over the interior-point solvers. In terms of iteration count, `filtermpec` is faster than `snopt`. This may be due to the fact that `filtermpec` is a second order method, while `snopt` is a quasi-Newton method. However, in terms of CPU time, `snopt` and `filtermpec` are roughly equivalent. We believe, that this is due to the faster linear algebra of `snopt`.

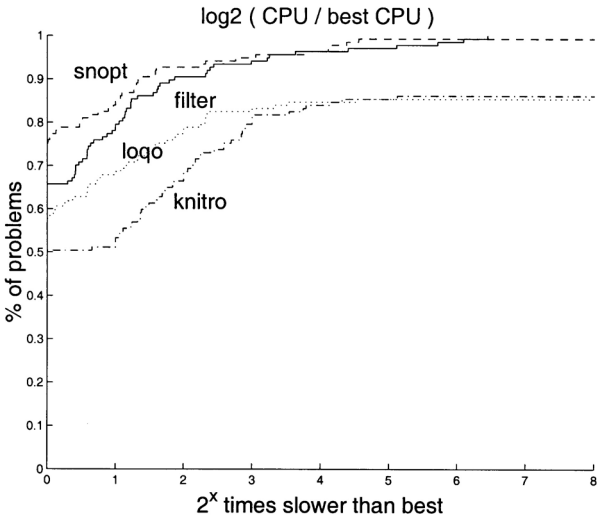

FIGURE 2 Performance plot (CPU time).

Downloaded by [LIU Libraries] at 08:10 02 May 2014

One reason for the poorer performance of interior-point methods could be the fact that the multipliers are generally larger than for the SQP methods. In [7] it is shown that SQP methods converge locally to a minimal multiplier. The same may not be true for interior-point methods. In Fig. 3, we show the relative size of the complementarity multiplier ξ. Let $\xi(s, p)$ be the complementarity multiplier of solver s on problem p. Figure 3 is obtained by plotting for every solver s,

$$\log_{10}\left(\frac{\xi(s, p)}{\max(1, \min_s\{\xi(s, p)\})}\right), \quad \forall p \in \text{ problem},$$

which is a measure of the size of the complementarity multiplier compared to the smallest complementarity multiplier (or 1). In this plot, we only include problems, where all solvers obtained the same solution and terminated successfully.

Figure 3 clearly indicates that the multiplier generated by the interior-point methods is several orders of magnitude larger than the multiplier generated by the SQP methods (which is minimal). It can be observed that SQP methods have smaller multipliers than interior-point methods. `loqo`'s multipliers are the largest, yet surprisingly it still outperforms `knitro`. The size of the multipliers is critical to the behavior of interior-point methods which follow a central path. Along the central path, the product of slacks and multipliers is controlled by the barrier parameter $\mu > 0$. Large multipliers usually imply small slacks, resulting in slow progress.

Finally, we comment on the nature of the stationary points found by `filtermpec`. Considering ξ in Appendix A shows that ξ is only large ($\mathcal{O}(\varepsilon^{-1})$) for `ex9.2.3`, `ralph1`, `ralphmod` and `scholtes5`. For all other problems, the limit is strongly stationary. Problems `ralph1` and `scholtes5` are known to have no strong stationary points. In this case, `filtermpec` converges to B-stationary point and the rate of convergence can be seen to be linear. We suspect that the same is true for `ralphmod` and `ex9.2.3`. It is remarkable that `filtermpec` does not converge to weaker (spurious) stationary points such as C-stationary points or M-stationary points.

5.3 Comparison to MPCC Solvers

In this section, the results for the NLP solvers are compared to some results published in the literature for special purpose MPCC solvers. The aim is to give an indication of the potential of using NLP solvers. This comparison has to be tentative, as we have no direct access to

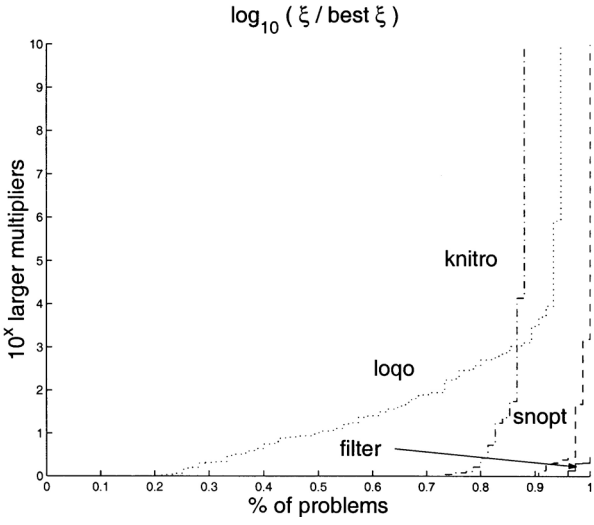

FIGURE 3 Relative size of complementarity multiplier ξ.

Downloaded by [LIU Libraries] at 08:10 02 May 2014

the MPCC solvers and all the experiments reported in the literature were run under different circumstances. Nevertheless, we believe that this comparison offers some useful insight into the efficiency of the NLP solvers.

The published numerical results considered in this section are the comparisons of Refs. [14, 16, 33]. Tables V–VII show the number of iterations required to solve the MPCCs compared to the results published in the literature. Since all solvers (with the exception of the interior-point solvers) perform roughly the same amount of work per iteration, this comparison seems to be fair.

Fukushima *et al.* [16] use a set of randomly generated problems (`flp4-1` to `flp4-4`). The problems we have generated have the same structure and size as those proposed in Ref. [16]. Note, however, that it is unlikely that the actual instances are identical. The random problem generator (written in MATLAB) used to generate `flp4-*` is available at www.mcs.anl.gov/∼leyffer/MacMPEC/tools/genflp4.m. Since the random problems differ from Ref. [16], any conclusion based on these instances is by necessity tentative.

It is clear, however, from Table V, that the SQP solver `filtermpec` compares very favorably with the MPCC method proposed in Ref. [16], often being an order of magnitude faster than the latter. Note also, that the interior-point solvers perform quite well on the problems. The multiplier ξ of the complementarity constraint is quite small for the interior-point solvers. The results for `snopt` are not as encouraging when compared to the other solvers. It is not immediately clear, why this should be. The only significant difference to the other NLP solvers is that `snopt` is a quasi-Newton method. However, at the solution, the null-space of `flp4-*` is zero, so second order convergence should be observed.

The next set of test results is taken from Ref. [14], where the iteration counts refer to algorithm AS. For the first four problems (`outrata31` to `outrata34`), we took starting point (a). For problem 9 (`nash1`), we have taken starting point (a). The number of iterations taken from Ref. [14] is 'KIN', the number of inner iterations or QP solves.

Again, it turns out that the SQP solvers in particular, are very competitive. While they do not always perform better than algorithm AS, especially, `filtermpec` has a smaller variation in the number of iterations than AS. The results for `loqo` are also very competitive, especially, if one takes into account that each iteration is far cheaper than either algorithm AS or an SQP method.

Dirkse and Ferris [33] test a total of five algorithms. In Table VII we only reproduce results for the two best representative, `bundle` and `pipa-b`. Ref. [33] uses slightly different problem names. These differences are summarized below.

[33]	`oz3`	`mss1`	`qvi`	`fjq*`	`hq1`
here	`bilevel3`	`gnash1*`	`nash1`	`outrata3*`	`stackelberg`

TABLE V Iteration count by Fukushima *et al.* [16]

No.	*Problem*	*Ref. [16]*	`filtermpec`	`loqo`	`knitro`	`snopt`
1	`stackelberg1`	22	4	14	37	3
2	`flp2`	48	5	29	27	13
3	`bilevel2`	31	1	18	50	10
4	`flp4-1`	50	3	19	17	66
4	`flp4-2`	50	4	21	16	88
4	`flp4-3`	51	4	20	16	96
4	`flp4-4`	50	4	23	20	161

Downloaded by [LIU Libraries] at 08:10 02 May 2014

TABLE VI Iteration count by Facchinei *et al.* [14]

No.	*Problem*	*Ref. [14]*	filtermpec	loqo	knitro	snopt
1	outrata31	27	8	18	38	9
2	outrata32	20	8	14	42	12
3	outrata33	17	7	19	38	9
4	outrata34	14	6	14	32	9
5	desilva	19	2	26	19	5
6	stackelberg1	5	4	14	37	3
7	bilevel1	28	6	15	36	5
8	gnash10	7	8	34	64	11
8	gnash11	7	8	31	55	11
8	gnash12	10	9	25	48	15
8	gnash13	7	13	24	44	18
8	gnash14	8	10	30	Fail	17
8	gnash15	5	18	25	86	8
8	gnash16	6	16	22	38	8
8	gnash17	6	16	21	38	12
8	gnash18	18	14	23	31	12
8	gnash19	20	10	24	44	83
9	nash1	7	1	33	17	0
10	bilevel2	51	1	18	50	10
11	bilevel3	7	7	22	36	14

One surprise from the comparison in Table VII is the fact that SQP methods appear to be as robust as the specially designed MPCC solvers. bundle has one failure which is solved in 32 iterations by filtermpec. snopt converges in only three iterations, but appears to get trapped at a nonstationary point or a local minimum (it cannot improve the current point). We believe this problem (ralphmod) is an unusual problem because it violates an MPCC constraint qualification. pipa-b has six failures, all of which are handled efficiently by the SQP methods.

TABLE VII Iteration count by Dirkse and Ferris [33]

Problem	bundle	pipa-b	filtermpec	loqo	knitro	snopt
bard3	1	Fail	4	22	48	9
desilva	9	145	2	26	19	5
outrata31	10	1000	8	18	38	9
outrata32	13	59	8	14	42	12
outrata33	8	35	7	19	38	9
outrata34	13	24	6	14	32	9
gauvin	13	18	7	20	29	10
stackelberg1	13	18	4	14	37	3
gnash10	14	29	8	34	64	11
gnash14	16	52	10	30	Fail	17
gnash19	11	21	10	24	44	83
bilevel1	2	1000	6	15	36	5
nash1	11	1000	1	33	17	0
ralphmod	Fail	1000	32	1000	109	3
tollmpec	7	Memory	16	1000	381	39

Downloaded by [LIU Libraries] at 08:10 02 May 2014

In addition, it is again clear that the SQP methods are at least competitive when compared to the specially developed MPCC solvers.

This comparison is clearly only a rough indication of the relative performance of the solvers. Comparing iteration counts does not adequately reflect the relative performance of the methods. For instance, interior-point methods solve a system of linear equations per iterations while SQP methods solve a quadratic program whose complexity is roughly the same as the subproblems solved in the algorithms in Refs. [14, 16, 33]. Nevertheless, the results presented here are informative: They indicate that SQP methods are better suited for solving MPCCs than interior-point solvers. Similarly, the results indicate that SQP methods are comparable in terms of speed with special purpose MPCC solvers. Clearly, more comparisons are needed, as well as further development of MPCC solvers to achieve a similar degree of robustness and speed as the NLP methods.

6 CONCLUSIONS

We have presented a large collection of MPCC test problems and compared the performance of standard NLP solvers on these problems. In particular, SQP methods seem very robust at solving MPCCs and their performance also compares favorably with special MPCC solvers on results published in the literature.

Interior-point methods are less robust, and fail on about 15% of problems. The reasons for these failures may not be due to the complementarity constraint; failures are also observed if the complementarity constraint is removed. In our view, these test demonstrate that SQP solvers can solve MPCCs efficiently and reliably. This underlines recent theoretical developments which have shown SQP methods to converge for MPCCs [see Refs. [1–8]].

We have demonstrated that the NLP approach to MPCCs provides a useful alternative to existing MPCC methods. The availability of large-scale solvers makes the approach suitable for large-scale MPCCs. In addition, NLP solvers allow more general constraints to be included that cannot be handled by special purpose MPCC solvers. These points make the NLP approach a practical and versatile alternative to MPCC solvers.

Downloaded by [LIU Libraries] at 08:10 02 May 2014

Acknowledgments

This work was supported EPSRC grant GR/M59549. We are also grateful for the opportunity to using Argonne's computing resources in carrying out our comparisons. We are grateful to Stefan Scholtes, Danny Ralph and Michael Ferris for many fruitful discussions on MPCCs. Finally, we gratefully acknowledge the insightful comments of an anonymous referee who greatly improved the article.

Many individuals provided help and input for the problem library. We are grateful to David Gay for sharing his AMPL expertise with us. Michal Kocvara provided help with the packaging problems. Francis Tin Loi supplied some challenging structural engineering problems. Finally, we have 'borrowed' test problems from a variety of sources but most notably from MPECLIB of Steven Dirkse.

References

[1] M.C. Ferris and J.S. Pang (1997). Engineering and economic applications of complementarity problems. *SIAM Review*, **39**(4), 669–713.

[2] Z.-Q. Luo, J.-S. Pang and D. Ralph (1996). *Mathematical Programs with Equilibrium Constraints*. Cambridge University Press, Cambridge, UK.

[3] J. Outrata, M. Kocvara and J. Zowe (1998). *Nonsmooth Approach to Optimization Problems with Equilibrium Constraints*. Kluwer Academic Publishers, Dordrecht.
[4] S. Leyffer (2000). MacMPEC: AMPL collection of MPECs. Webpage, www.mcs.anl.gov/ leyffer/MacMPEC/.
[5] Y. Chen and M. Florian (1995). The nonlinear bilevel programming problem: Formulations, regularity and optimality conditions. *Optimization*, **32**, 193–209.
[6] H. Scheel and S. Scholtes (2000). Mathematical program with complementarity constraints: Stationarity, optimality and sensitivity. *Mathematics of Operations Research*, **25**, 1–22.
[7] R. Fletcher, S. Leyffer, D. Ralph and S. Scholtes (2002). Local convergence of SQP methods for mathematical programs with equilibrium constraints. Numerical Analysis Report NA/209, Department of Mathematics, University of Dundee, Dundee, UK, May 2002.
[8] M. Anitescu (2000). *On Solving Mathematical Programs with Complementarity Constraints as Nonlinear Programs*. Preprint ANL/MCS-P864-1200. MCS Division, Argonne National Laboratory, Argonne, IL. Submitted to *SIAM J. Optimization*.
[9] P.E. Gill, W. Murray and M.A. Saunders (1997). SNOPT: An SQP algorithm for large-scale constrained optimization. Report NA 97-2, Dept. of Mathematics, University of California, San Diego, November 1997.
[10] J.F. Bard (1988). Convex two-level optimization. *Mathematical Programming*, **40**(1), 15–27.
[11] A.R. Conn, N.I.M. Gould and Ph.L. Toint (1996). Numerical experiments with the Lancelot package (Release A) for large-scale nonlinear optimization. *Mathematical Programming*, **73**(1), 73–110.
[12] M.C. Ferris and C. Kanzow (1999). Complementarity and related problems: A survey. Mathematical Programming Technical Report 98-17, University of Wisconsin, August 1999.
[13] M.C. Ferris and F. Tin-Loi (1998). Nonlinear programming approach for a class of inverse problems in elastoplasticity. *Structural Engineering and Mechanics*, **6**, 857–870.
[14] F. Facchinei, H. Jiang, and L. Qi (1999). A smoothing method for mathematical programs with equilibrium constraints. *Mathematical Programming*, **85**, 107–134.
[15] H. Jiang and D. Ralph (2000). Smooth SQP methods for mathematical programs with nonlinear complementarity constraints. *SIAM J. Optimization*, **10**, 779–808.
[16] M. Fukushima, Z.-Q. Luo and J.-S. Pang (1998). A globally convergent sequential quadratic programming algorithm for mathematical programs with linear complementarity constraints. *Computational Optimization and Applications*, **10**(1), 5–34.
[17] S. Leyffer (2002). The penalty interior point method fails to converge. Numerical Analysis Report NA/208, Department of Mathematics, University of Dundee, Dundee, UK, February 2002. Revised Sept. 2003.
[18] S.P. Dirkse and M.C. Ferris (1999). Modelling and solution environments for MPEC: GAMS & MATLAB. In: M. Fukushima and L. Qi (Eds.), *Reformulation: Nonsmooth, Piecewise Smooth, Semismooth and Smoothing Methods*. Kluwer Academic, Dordrecht.
[19] S.P. Dirkse, M.C. Ferris and A. Meeraus (2002). Mathematical programs with equilibrium Constraints: Automatic reformulation and solution via constraint optimization. Technical Report NA-02/11, Oxford University Computing Laboratory, July 2002.
[20] R. Fletcher and S. Leyffer (2002). Nonlinear programming without a penalty function. *Mathematical Programming*, **91**, 239–270.
[21] R. Fourer, D.M. Gay and B.W. Kernighan (2003). *AMPL: A Modelling Language for Mathematical Programming*. Books/Cole – Thomson Learning, 2nd ed.
[22] S. Leyffer (2003). Complementarity constraints as nonlinear equations: Theory and numerical experience. Technical report, Argonne National Laboratory, 2003.
[23] B. Fourer, M.C. Ferris and D.M. Gay (1999). Expressing complementarity problems and communicating them to solvers. *SIAM Journal on Optimization*, **9**, 991–1009.
[24] I. Bongartz, A.R. Conn, N.I.M. Gould and Ph.L. Toint (1995). CUTE: Constrained and unconstrained testing enviroment. *ACM Transactions on Mathematical Software*, (21), 123–160.
[25] S.C. Billups, S.P. Dirkse and M.C. Ferris (1997). A comparison of large scale mixed complementarity problem solvers. *Computational Optimization and Applications*, **7**, 3–25.
[26] J. Czyzyk, M. Mesnier and J. Moré (1998). The NEOS server. *IEEE Journal on Computational Science and Engineering*, **5**, 68–75. Try it at www-neos.mcs.anl.gov/neos/!
[27] R. Fletcher, S. Leyffer and Ph.L. Toint (1998). On the global convergence of an SLP-filter algorithm. Numerical Analysis Report NA/183, University of Dundee, UK, August 1998.
[28] R.H. Byrd, M.E. Hribar and J. Nocedal (1999). An interior point algorithm for large scale nonlinear programming. *SIAM J. Optimization*, **9**(4), 877–900.
[29] R.J. Vanderbei and D.F. Shanno (1999). An interior point algorithm for nonconvex nonlinear programming. *COAP*, **13**, 231–252.
[30] A. Raghunathan and L.T. Biegler (2002). Barrier methods for mathematical programs with complementarity constraints (MPCCs). Technical report, Carnegie Mellon University, Department of Chemical Engineering, Pittsburgh, PA, December 2002.
[31] H. Benson, D.F. Shanno and R.V.D. Vanderbei (2002). Interior-point methods for nonconvex nonlinear programming: Complementarity constraints. Technical Report ORFE 02-02, Princeton University, Operations Research and Financial Engineering.
[32] E.D. Dolan and Jorge Moré (2000). *Benchmarking Optimization Software with Cops*. Technical Report MCS-TM-246, MCS Division, Argonne National Laboratory, Argonne IL, November 2000.

Downloaded by [LIU Libraries] at 08:10 02 May 2014

[33] S.P. Dirkse and M.C. Ferris (1999). Modeling and solution environments for mpec: Gams & matlab. In: M. Fukushima and L. Qi (Eds.), *Reformulation: Nonsmooth, Piecewise Smooth, Semismooth and Smoothing Methods*, pp. 127–148. Kluwer Academic Publishers.
[34] Z.-Q. Luo, J.-S. Pang, D. Ralph and S.-Q. Wu (1996). Exact penalization and stationarity conditions of mathematical programs with equilibrium constraints. *Mathematical Programming*, **75**(1), 19–76.

A TEST PROBLEM CHARACTERISTICS

This section gives details of the characteristics of all MacMPEC test problems. In the tables below, n is the number of variables (excluding slacks), m is the number of general constraints (excluding simple bounds), p is the number of complementarity conditions, n_{NLP} is the number of variables plus slacks and k is the dimension of the null-space. The next three columns give the degrees of degeneracy of the upper level (d_1), the lower level (d_2) problem and the degree of mixed degeneracy (d_m), see Ref. [34]. iter is the number of iterations by `filtermpec`, compl is the final complementarity error, i.e. $z_1^{\mathrm{T}} z_2$ and ξ is the multiplier of the complementarity constraint. For the latter two columns, if no value is given, then the solution returned an exact zero.

Downloaded by [LIU Libraries] at 08:10 02 May 2014

Downloaded by [LIU Libraries] at 08:10 02 May 2014

Name	n	m	p	n_{NLP}	k	d_1	d_2	d_m	*iter*	*compl*	ξ
bar-truss-3	35	34	6	35	0	13	0	0	14		−1.45
bard1	5	4	3	8	0	3	0	0	1		−0.762
bard1m	6	4	3	9	0	4	0	0	1		−0.762
bard2	12	9	3	15	0	2	1	0	2		
bard2m	12	9	3	15	0	2	1	0	1		
bard3	6	5	1	7	0	2	0	0	4		
bard3m	6	5	3	9	0	2	0	0	4		−1.09
bem-milanc30-s	3436	3433	1464	3436	2	1698	1	0	56		−119.
bilevel1	10	9	6	12	0	4	0	0	6		−0.150
bilevel2	16	13	8	20	1	5	0	1	1		
bilevel3	11	10	3	11	0	5	0	0	7		−1.09
bilin	8	7	6	14	0	3	0	1	4		−50.0
dempe	3	2	1	4	0	0	0	0	58		−0.571E−05
design-cent-1	12	11	3	15	0	6	0	0	4		−2.17
design-cent-2	13	15	3	16	1	6	0	0	11		−4.39
design-cent-3	15	11	3	18	0	10	0	1	24		
design-cent-4	22	20	8	30	1	12	0	1	4		−0.845
desilva	6	4	2	8	0	2	0	2	2		
df1	2	3	1	3	1	1	0	1	2		
ex9.1.1	13	12	5	13	0	7	0	2	3		−0.500
ex9.1.10	11	9	3	11	0	5	0	2	1		
ex9.1.2	8	7	2	8	0	4	0	0	1		
ex9.1.3	23	21	6	23	0	15	0	2	4		−34.0
ex9.1.4	8	7	2	8	0	3	0	1	4		
ex9.1.5	13	12	5	13	0	10	0	2	1		
ex9.1.6	14	13	6	14	0	7	0	0	3		−1.00
ex9.1.7	17	15	6	17	0	8	0	2	4		−34.0
ex9.1.8	11	9	3	11	0	5	0	2	1		
ex9.1.9	12	11	5	12	0	5	0	1	2		−0.444

(*continued*)

Downloaded by [LIU Libraries] at 08:10 02 May 2014

Continued

Name	n	m	p	n_{NLP}	k	d_1	d_2	d_m	*iter*	*compl*	ξ
ex9.2.1	10	9	4	10	0	3	1	0	6		−2.42
ex9.2.2	9	8	3	9	0	3	0	1	27	0.297E−12	−0.303E+07
ex9.2.3	14	13	4	14	0	5	1	0	2		
ex9.2.4	8	7	2	8	0	3	0	0	7		−1.000
ex9.2.5	8	7	3	8	0	3	0	0	8		−6.00
ex9.2.6	16	12	6	16	0	8	0	4	1		−0.500
ex9.2.7	10	9	4	10	0	3	1	0	6		−2.42
ex9.2.8	6	5	2	6	0	2	1	0	1		−0.500
ex9.2.9	9	8	3	9	0	7	0	1	1		
gauvin	3	2	2	5	0	1	0	0	7		−0.250
gnash10	13	12	8	13	1	0	0	0	8		−0.142
gnash11	13	12	8	13	1	0	0	0	8		−0.918E−01
gnash12	13	12	8	13	1	0	0	0	9		−0.397E−01
gnash13	13	12	8	13	1	0	0	0	13		−0.149E−01
gnash14	13	12	8	13	1	0	0	0	10		−0.199E−02
gnash15	13	12	8	13	0	3	0	0	18		−7.65
gnash16	13	12	8	13	0	4	0	0	16		−1.95
gnash17	13	12	8	13	1	4	0	0	16		−1.67
gnash18	13	12	8	13	1	4	0	0	14		−12.7
gnash19	13	12	8	13	0	2	0	0	10		−2.80
hakonsen	9	8	4	9	0	2	0	0	10		−0.390
hs044-i	20	14	10	26	0	7	0	2	6		−5.69
incid-set1-08	117	119	49	117	0	52	0	1	40		
incid-set1-16	485	491	225	485	0	232	0	5	34		
incid-set1-32	1989	2003	961	1989	0	188	26	5	112		
incid-set1c-08	117	126	49	117	0	64	0	4	41		
incid-set1c-16	485	506	225	485	1	250	0	13	24		
incid-set1c-32	1989	2034	961	1989	2	145	21	0	108		
incid-set2-08	117	119	49	166	3	42	7	0	39		
incid-set2-16	485	491	225	710	3	214	11	0	26		
incid-set2-32	1989	2003	961	2950	5	940	24	0	69		

Downloaded by [LIU Libraries] at 08:10 02 May 2014

incid-set2c-08	117	126	49	166	2	46	6	0	33		
incid-set2c-16	485	506	225	710	0	219	12	0	36		
incid-set2c-32	1989	2034	961	2950	1	937	24	0	66		
jr1	2	1	1	3	1	0	0	0	1		
jr2	2	1	1	3	0	0	0	0	6		−2.00
kth1	2	1	1	2	0	0	1	0	1		
kth2	2	1	1	2	1	0	0	0	2		
kth3	2	1	1	2	0	0	0	0	4		−1.00
liswet1-050	152	103	50	202	1	52	0	0	1		
liswet1-100	302	203	100	402	1	102	0	0	1		
liswet1-200	602	403	200	802	1	202	0	0	1		
nash1	6	4	2	8	2	4	0	0	1	0.153E−08	
outrata31	5	4	4	9	0	0	1	0	8		−0.164
outrata32	5	4	4	9	1	0	0	0	8		−0.168
outrata33	5	4	4	9	1	1	0	0	7		−0.714
outrata34	5	4	4	9	1	1	0	0	6		−2.07
pack-comp1-08	107	121	49	156	0	113	0	0	6	0.728E−06	
pack-comp1-16	467	511	225	692	4	268	0	2	8		
pack-comp1-32	1955	2107	961	2916	6	1063	0	3	49		
pack-comp1c-08	107	128	49	156	0	120	0	0	6	0.728E−06	
pack-comp1c-16	467	526	225	692	1	267	0	1	5		
pack-comp1c-32	1955	2138	961	2916	3	1062	0	2	7		
pack-comp2-08	107	121	49	156	5	62	0	2	9		
pack-comp2-16	467	511	225	692	4	268	0	2	33		
pack-comp2-32	1955	2107	961	2916	6	1066	2	2	36		
pack-comp2c-08	107	128	49	156	1	62	0	2	6		
pack-comp2c-16	467	526	225	692	4	268	0	2	13		
pack-comp2c-32	1955	2138	961	2916	13	1060	0	2	15		
pack-rig1-08	87	86	32	108	7	48	0	0	12		
pack-rig1-16	380	379	158	485	6	207	0	0	16		
pack-rig1-32	1622	1621	708	2087	4	789	0	0	58		−12.6
pack-rig1c-08	87	93	32	108	4	46	0	0	8	0.153E−07	
pack-rig1c-16	380	394	158	485	4	205	0	0	7	0.390E−06	
pack-rig1c-32	1622	1652	708	2087	0	924	0	0	22	0.310E−06	

(*continued*)

Downloaded by [LIU Libraries] at 08:10 02 May 2014

Continued

Name	n	m	p	n_{NLP}	k	d_1	d_2	d_m	*iter*	*compl*	ξ
pack-rig1p-08	105	104	47	126	3	50	2	0	14		
pack-rig1p-16	445	444	203	550	3	231	2	0	23		
pack-rig1p-32	1855	1854	861	2320	4	868	2	0	135		−12.6
pack-rig2-08	85	84	30	106	6	43	0	0	12		
pack-rig2-16	375	374	149	480	0	207	0	0	8	0.706E−06	
pack-rig2-32	1580	1579	661	2045	0	893	0	0	14	0.482E−06	
pack-rig2c-08	85	91	30	106	2	43	0	0	6	0.477E−08	
pack-rig2c-16	375	389	149	480	0	221	0	0	7	0.379E−06	
pack-rig2c-32	1580	1610	661	2045	0	928	0	0	11	0.399E−06	
pack-rig2p-08	103	102	45	124	3	45	2	0	9	0.369E−13	
pack-rig2p-16	436	435	194	541	1	213	1	0	15		
pack-rig2p-32	1808	1807	814	2273	2	942	1	0	28		
portfl1	87	25	12	87	6	6	0	0	6		−0.896
portfl2	87	25	12	87	5	7	0	0	5		−0.682
portfl3	87	25	12	87	12	6	0	0	4		−31.1
portfl4	87	25	12	87	15	5	0	0	4		−116.
portfl6	87	25	12	87	13	5	0	0	4		−54.4
qpec-100-1	105	102	100	205	0	75	2	0	5		−4.77
qpec-100-2	110	102	100	210	0	57	3	0	6		−2.12
qpec-100-3	110	104	100	210	0	33	2	0	9		−125.
qpec-100-4	120	104	100	220	0	62	6	0	5		−3.66

Downloaded by [LIU Libraries] at 08:10 02 May 2014

qpec-200-1	210	204	200	410	0	153	2	0	10		−158.
qpec-200-2	220	204	200	420	0	115	4	0	10		−5.36
qpec-200-3	220	208	200	420	0	48	6	0	12		−35.5
qpec-200-4	240	208	200	440	0	135	10	0	6		−4.96
qpec1	30	20	20	40	0	10	10	0	3		
qpec2	30	20	20	40	0	0	10	0	2		−0.667
ralph1	2	1	1	3	0	0	0	0	27	0.383E−12	−0.538E+06
ralph2	2	1	1	2	1	0	0	0	11	0.240E−06	−2.00
ralphmod	104	100	100	204	1	78	0	0	32		−0.136E+06
scholtes1	3	1	1	4	0	1	0	0	4		
scholtes2	3	1	1	4	0	0	1	0	2		
scholtes3	2	1	1	2	0	0	0	0	4		−1.00
scholtes4	3	3	1	3	0	0	0	0	25	0.609E−13	−0.270E+07
scholtes5	3	2	2	3	2	0	0	0	1		
sl1	8	5	3	11	1	5	0	2	1		
stackelberg1	3	2	1	3	1	0	0	0	4		
tap-09	86	68	32	118	0	67	0	7	29		−40.0
tap-15	194	167	83	277	0	164	0	37	21		−91.9
tollmpec	2403	2376	1748	4151	0	1588	1	86	16		−8.46
tollmpec1	2403	2376	1748	4151	0	1608	0	85	16		−165.
water-FL	213	160	44	213	4	53	0	0	308		−0.174E+05
water-net	66	50	14	66	3	15	0	0	83		

B COMPARISON OF ITERATION COUNT AND CPU TIME

Name	filtermpec *iter*	filtermpec *cpu*	loqo *iter*	loqo *cpu*	knitro *iter*	knitro *cpu*	snopt *iter*	snopt *cpu*
bar-truss-3	14	0.01	27	0.01	120	0.04	10	0.00
bard1	1	0.00	13	0.00	25	0.00	0	0.00
bard1m	1	0.00	13	0.00	27	0.01	4	0.00
bard2	2	0.00	32	0.00	34	0.01	4	0.00
bard2m	1	0.00	28	0.00	34	0.01	4	0.00
bard3	4	0.00	22	0.00	48	0.00	9	0.00
bard3m	4	0.00	23	0.00	66	0.01	10	0.00
bem-milanc30-s	56	409.48	556	72.89	139	43.89	479	604.00
bilevel1	6	0.00	15	0.00	36	0.01	5	0.01
bilevel2	1	0.00	18	0.00	50	0.01	10	0.00
bilevel3	7	0.01	22	0.00	36	0.00	14	0.00
bilin	4	0.00	71	0.01	48	0.01	4	0.00
dempe	58	0.00	16	0.00	1000	fail	0	0.00
design-cent-1	4	0.00	14	0.00	24	0.00	5	0.00
design-cent-2	11	0.01	26	0.00	25	0.00	8	0.00
design-cent-3	24	0.01	83	0.04	35	0.01	17	0.01
design-cent-4	4	0.00	1000	fail	23	0.01	5	0.00
desilva	2	0.00	26	0.00	19	0.01	5	0.00
df1	2	0.00	29	0.00	19	0.01	7	0.00
ex9.1.1	3	0.00	15	0.00	33	0.01	3	0.00
ex9.1.10	1	0.00	31	0.00	34	0.01	3	0.00
ex9.1.2	1	0.00	13	0.00	28	0.01	3	0.00
ex9.1.3	4	0.00	39	0.01	25	0.01	0	0.00
ex9.1.4	4	0.00	38	0.00	31	0.00	3	0.00
ex9.1.5	1	0.00	11	0.00	17	0.00	0	0.00
ex9.1.6	3	0.00	14	0.00	31	0.00	3	0.00
ex9.1.7	4	0.00	74	0.01	26	0.01	0	0.00
ex9.1.8	1	0.00	31	0.00	34	0.01	3	0.00
ex9.1.9	2	0.00	20	0.00	27	0.00	4	0.00
ex9.2.1	6	0.00	13	0.00	29	0.01	0	0.00
ex9.2.2	27	0.01	279	0.03	77	0.01	33	0.00
ex9.2.3	2	0.00	38	0.01	46	0.01	0	0.00
ex9.2.4	7	0.00	11	0.00	19	0.00	5	0.00
ex9.2.5	8	0.00	16	0.00	25	0.01	8	0.01
ex9.2.6	1	0.00	25	0.00	21	0.00	1	0.00
ex9.2.7	6	0.00	13	0.00	29	0.00	0	0.00
ex9.2.8	1	0.00	11	0.00	25	0.00	0	0.00
ex9.2.9	1	0.00	13	0.00	28	0.00	2	0.00
gauvin	7	0.00	20	0.00	29	0.00	10	0.00
gnash10	8	0.00	34	0.00	64	0.02	11	0.00
gnash11	8	0.00	31	0.01	55	0.01	11	0.00
gnash12	9	0.00	25	0.00	48	0.01	15	0.01
gnash13	13	0.00	24	0.00	44	0.01	18	0.01
gnash14	10	0.00	30	0.00	1234	fail	17	0.01
gnash15	18	0.01	25	0.00	86	0.02	8	0.00
gnash16	16	0.00	22	0.00	38	0.01	8	0.00
gnash17	16	0.01	21	0.00	38	0.01	12	0.01
gnash18	14	0.00	23	0.00	31	0.01	12	0.01
gnash19	10	0.00	24	0.00	44	0.01	83	0.03

(*continued*)

Downloaded by [LIU Libraries] at 08:10 02 May 2014

Continued

Name	filtermpec *iter/cpu*		loqo *iter/cpu*		knitro *iter/cpu*		snopt *iter/cpu*	
hakonsen	10	0.00	21	0.00	50	0.00	64	0.01
hs044-i	6	0.00	25	0.01	33	0.01	28	0.01
incid-set1-08	40	0.61	23	0.20	462	2.76	35	0.42
incid-set1-16	34	5.75	49	2.62	1000	fail	1000	fail
incid-set1-32	112	429.25	1000	fail	1000	fail	990	1078.66
incid-set1c-08	41	0.71	26	0.21	384	2.44	15	0.14
incid-set1c-16	24	4.79	78	3.91	1000	fail	70	3.90
incid-set1c-32	108	613.12	1000	fail	1000	fail	255	208.71
incid-set2-08	39	0.56	34	0.28	981	4.77	58	0.24
incid-set2-16	26	5.35	1000	fail	1000	fail	85	4.06
incid-set2-32	69	578.84	1000	fail	1000	fail	66	60.74
incid-set2c-08	33	0.44	32	0.26	209	1.31	46	0.24
incid-set2c-16	36	9.35	102	5.31	1000	fail	36	1.79
incid-set2c-32	66	420.87	1000	fail	1000	fail	172	115.22
jr1	1	0.00	9	0.00	9	0.00	3	0.00
jr2	6	0.00	11	0.00	9	0.00	7	0.00
kth1	1	0.00	9	0.00	8	0.00	1	0.00
kth2	2	0.00	11	0.00	7	0.00	4	0.00
kth3	4	0.00	10	0.00	22	0.00	6	0.00
liswet1-050	1	0.02	20	0.03	28	0.08	59	0.36
liswet1-100	1	0.08	24	0.06	32	0.25	84	1.25
liswet1-200	1	0.35	31	0.14	27	0.45	82	3.30
nash1	1	0.00	33	0.00	17	0.00	0	0.00
outrata31	8	0.00	18	0.00	38	0.00	9	0.00
outrata32	8	0.00	14	0.00	42	0.00	12	0.00
outrata33	7	0.00	19	0.01	38	0.01	9	0.00
outrata34	6	0.00	14	0.00	32	0.00	9	0.00
pack-comp1-08	6	0.07	43	0.24	27	0.18	4	0.03
pack-comp1-16	8	1.66	52	1.81	91	4.02	22	1.11
pack-comp1-32	49	1813.46	101	20.74	1000	fail	43	50.00
pack-comp1c-08	6	0.07	34	0.16	40	0.24	8	0.04
pack-comp1c-16	5	1.14	78	2.58	70	3.34	9	0.76
pack-comp1c-32	7	71.77	54	9.04	106	39.15	18	16.78
pack-comp2-08	9	0.10	36	0.20	42	0.25	15	0.10
pack-comp2-16	33	7.80	69	2.48	705	33.25	78	2.54
pack-comp2-32	36	1027.19	97	19.53	207	69.78	121	337.57
pack-comp2c-08	6	0.08	23	0.11	40	0.29	7	0.04
pack-comp2c-16	13	2.65	47	1.46	67	3.42	29	1.13
pack-comp2c-32	15	434.56	68	12.55	69	27.07	24	17.46
pack-rig1-08	12	0.10	40	0.16	22	0.11	17	0.07
pack-rig1-16	16	1.49	59	1.47	70	2.00	43	0.96
pack-rig1-32	58	259.83	79	12.27	1000	fail	48	101.87
pack-rig1c-08	8	0.06	25	0.10	26	0.13	12	0.04
pack-rig1c-16	7	0.58	50	1.22	45	1.37	27	0.91
pack-rig1c-32	22	42.20	104	15.07	88	13.69	30	14.93
pack-rig1p-08	14	0.11	34	0.14	43	0.25	14	0.08
pack-rig1p-16	23	2.51	50	1.30	79	2.69	29	1.31
pack-rig1p-32	135	671.76	68	9.83	1000	fail	35	24.29
pack-rig2-08	12	0.08	33	0.14	28	0.13	16	0.05
pack-rig2-16	8	0.56	1000	fail	1000	fail	6	0.27
pack-rig2-32	14	25.60	1000	fail	1000	fail	37	19.30

(*continued*)

Downloaded by [LIU Libraries] at 08:10 02 May 2014

Continued

Name	filtermpec *iter*	filtermpec *cpu*	loqo *iter*	loqo *cpu*	knitro *iter*	knitro *cpu*	snopt *iter*	snopt *cpu*
pack-rig2c-08	6	0.03	28	0.11	31	0.14	10	0.03
pack-rig2c-16	7	0.50	1000	fail	1000	fail	6	0.33
pack-rig2c-32	11	22.35	1000	fail	1000	fail	7	6.47
pack-rig2p-08	9	0.08	35	0.15	25	0.14	15	0.06
pack-rig2p-16	15	1.52	55	1.52	58	1.91	22	0.68
pack-rig2p-32	28	131.05	1000	fail	401	87.31	28	26.26
portfl1	6	0.02	1000	fail	26	0.09	7	0.02
portfl2	5	0.02	169	0.20	21	0.08	7	0.02
portfl3	4	0.01	1000	fail	20	0.08	8	0.02
portfl4	4	0.01	1000	fail	20	0.08	7	0.03
portfl6	4	0.01	210	0.23	23	0.08	7	0.02
qpec-100-1	5	0.22	30	0.83	41	1.58	21	0.43
qpec-100-2	6	0.37	34	1.84	66	2.76	67	0.95
qpec-100-3	9	0.77	28	1.14	48	2.10	74	1.10
qpec-100-4	5	0.36	32	1.72	63	2.71	22	0.50
qpec-200-1	10	2.02	44	10.23	47	14.22	37	3.38
qpec-200-2	10	8.01	39	18.87	77	24.06	28	3.72
qpec-200-3	12	7.92	41	9.80	50	20.40	24	9.11
qpec-200-4	6	2.52	58	28.57	89	29.32	24	2.65
qpec1	3	0.00	10	0.00	22	0.00	5	0.00
qpec2	2	0.00	266	0.05	44	0.02	68	0.05
ralph1	27	0.00	271	0.01	9	0.00	33	0.00
ralph2	11	0.00	21	0.00	15	0.00	10	0.00
ralphmod	32	0.76	1000	fail	109	4.88	3	0.14
scholtes1	4	0.00	20	0.00	42	0.01	5	0.00
scholtes2	2	0.00	11	0.00	16	0.00	3	0.01
scholtes3	4	0.00	29	0.00	10	0.01	0	0.00
scholtes4	25	0.01	1000	fail	11	0.00	36	0.00
scholtes5	1	0.00	10	0.00	8	0.00	3	0.00
sl1	1	0.00	71	0.01	33	0.01	3	0.01
stackelberg1	4	0.00	14	0.00	37	0.01	3	0.00
tap-09	29	0.09	50	0.07	42	0.18	44	0.17
tap-15	21	0.31	57	0.33	71	1.47	45	0.94
tollmpec	16	164.09	1000	fail	381	74.44	39	352.85
tollmpec1	16	161.54	1000	fail	449	80.97	60	548.23
water-FL	308	3.72	1000	fail	37	fail	95	0.30
water-net	83	0.15	1000	fail	1000	fail	308	0.32

Downloaded by [LIU Libraries] at 08:10 02 May 2014